

HACKATHON 1

LLM-Based Intelligent Customer Support Chatbot with Knowledge Retrieval

TEAM 13

Roll Number	Team Member
2420030218	M. Vennela Naga Sai
2420090073	R. Gayatri Satya Sai Pravallika

1. Project Overview

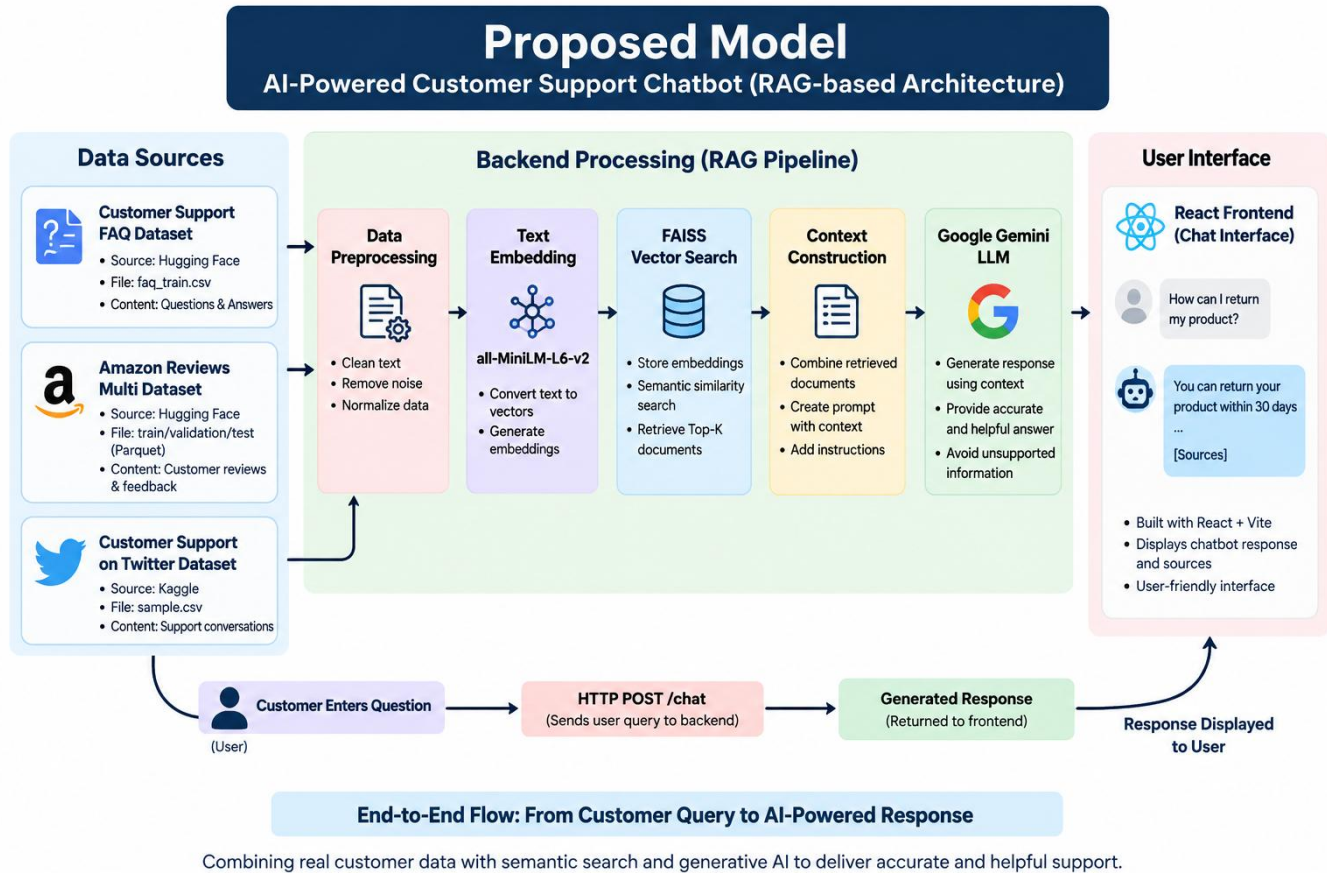
The LLM-Based Intelligent Customer Support Chatbot is an AI-powered customer-support system that combines semantic information retrieval with a Large Language Model (LLM). The project uses Retrieval-Augmented Generation (RAG) so that customer questions are answered using relevant information retrieved from a customer-support knowledge base. Instead of asking the language model to answer entirely from its internal knowledge, the system first retrieves relevant support information and then provides that information to the LLM as context.

2. Problem Statement

Traditional customer-support systems often depend on keyword matching, static FAQs, or manual browsing of support documentation. These approaches may fail when customers describe the same issue using different words. On the other hand, a standalone LLM can generate fluent answers but may introduce information that is not present in the support knowledge base. The project addresses this problem by combining semantic retrieval, vector search, and controlled LLM generation.

3. Proposed Model

The proposed model is a Retrieval-Augmented Generation pipeline. Customer-support records are converted into vector embeddings using the all-MiniLM-L6-v2 sentence transformer model. These embeddings are indexed using FAISS. When a customer submits a question, the same embedding model converts the question into a vector and FAISS retrieves the most similar support records. The retrieved information is then inserted into a controlled prompt and passed to Google Gemini to generate the final response.



4. Proposed Architecture

Customer Question → Query Embedding → FAISS Semantic Retrieval → Top-K Relevant Support Documents → Context Construction → Google Gemini → Grounded Response → React UI

Offline knowledge preparation:

1. Load the customer-support FAQ dataset.
2. Process and normalize the support text used by the retrieval pipeline.
3. Generate sentence embeddings using all-MiniLM-L6-v2.
4. Build a FAISS IndexFlatL2 vector index.
5. Store the vector index and corresponding FAQ metadata.

Online question-answering pipeline:

6. Receive the customer's question through the FastAPI /chat endpoint.
7. Generate an embedding for the incoming question.
8. Search the FAISS index using L2 nearest-neighbor search.
9. Retrieve the top-K relevant support records.
10. Construct a prompt containing the customer's question and retrieved context.
11. Send the prompt to Google Gemini.
12. Return the generated answer and retrieved source information to the frontend.

5. Technology Stack

Layer	Technology	Purpose
Programming	Python	Backend, retrieval and data processing
Embeddings	Sentence Transformers	Semantic representation of text
Embedding Model	all-MiniLM-L6-v2	Query and document embeddings
Vector Search	FAISS	Similarity search over embeddings
Index	IndexFlatL2	L2 nearest-neighbor retrieval
RAG	Custom RAG pipeline	Retrieve context before generation
LLM	Google Gemini	Natural-language response generation
Backend	FastAPI	REST API and application backend
Frontend	React + Vite	Interactive customer-support interface
Data	Pandas / NumPy	Data handling and numerical processing
Version Control	Git / GitHub	Source code and documentation

6. Datasets

Primary Knowledge Base

Customer Support FAQ dataset: MakTek/Customer_support_faqs_dataset from Hugging Face. The project contains the FAQ question and answer information used by the primary retrieval pipeline.

Main fields used by the project include:

- question
- answer
- source
- text / processed text

Supplementary Datasets

- Amazon Reviews Multi — English train, validation and test Parquet files.
- Customer Support on Twitter — collected support-conversation sample.

These supplementary datasets are stored and documented in the repository. They should not be described as fully integrated into the live FAISS index unless that integration is explicitly implemented.

7. Core Components

Component	Role
embeddings.py	Generates embeddings using all-MiniLM-L6-v2.
faiss_index.py	Builds and saves the FAISS vector index and metadata.
retriever.py	Embeds a query and retrieves top-K records from FAISS.
prompt_builder.py	Constructs the grounded prompt from retrieved context.

llm.py	Sends the prompt to Google Gemini.
rag_pipeline.py	Connects retrieval, prompt construction and generation.
main.py	Exposes the chatbot through FastAPI.

8. Grounding and Prompt Strategy

The prompt instructs the LLM to answer using only the supplied customer-support information, avoid unsupported policies or procedures, keep responses concise, and state when the available information is insufficient. It also instructs the model not to expose internal implementation details such as FAISS, embeddings, RAG, or datasets.

9. Frontend and Backend Integration

The React + Vite frontend sends customer questions to the FastAPI backend using HTTP. The backend executes the RAG pipeline and returns a JSON response containing the generated answer and retrieved source records. The frontend then displays the result to the customer.

10. Product-Aware Extension

The project architecture is designed to support product-aware customer service. In the extended design, a customer selects a purchased product and submits a question. The product identifier is used as metadata to filter candidate support records before FAISS semantic ranking. This enables product-specific retrieval. Reliable implementation requires verified product metadata to be associated with support records; products should not be assigned arbitrarily.

11. Security and Responsible Implementation

- API keys are stored in a local .env file and should not be committed to GitHub.
- The chatbot should not invent policies, prices, dates or procedures.
- Retrieved context should be treated as the knowledge boundary for grounded responses.
- Dataset licenses and usage requirements should be reviewed before production deployment.

12. Project Objectives

- Build a semantic customer-support retrieval system.
- Integrate vector search with an LLM using RAG.
- Generate grounded and context-aware customer-support responses.
- Expose the system through a FastAPI backend.
- Provide an interactive React frontend.
- Evaluate retrieval and response-generation quality using appropriate metrics.
- Develop an architecture that can be extended to product-aware customer support.

13. Summary

The project is an end-to-end LLM-based customer-support system that combines semantic retrieval, FAISS vector search, Retrieval-Augmented Generation, Google Gemini, FastAPI, and React. Its central design principle is to retrieve relevant support knowledge first and then use that information to guide natural-language generation.

